
EECS 545 Machine Learning

Progress Report

GNN on Traveling Salesman Problem

Pingbang Hu

Department of Computer Science
University of Michigan
pbb@umich.edu

Jonathan Moore

Department of Computer Science
University of Michigan
moorej@umich.edu

Yi Zhou

Department of Math
University of Michigan
yizho@umich.edu

Shubham Kumar Pandey

Department of Statistics
University of Michigan
pandeysk@umich.edu

Anuraag Ramesh

Department of Statistics
University of Michigan
anuraagr@umich.edu

Abstract

This is our progress report for the EECS 545 course project. Graph neural network is a rising concept in computer science that has demonstrated significant potential in many areas. Our project aims to further demonstrate its potential by applying it to the classic traveling salesman problem. After generating the dataset, we will first use imitation learning to obtain a pretty good path, which will then be inputted into reinforcement learning to obtain the optimal solution. Since imitation learning converges very fast, we expect our run time to be promising compared to tradition exact solutions.

In this progress report, we will describe and explain our proposed method, share important related works, show our difference with the existing works, present our experimental results, and set up weekly goals for future development.

Keywords: Travelling salesman problem, Graph Neural Network, Imitation Training, Reinforcement Learning, Transformer, Integer Programming, Embedding learning, Link Prediction, Combinatorial Optimization, Learning theory.

1 Introduction

The travelling salesman problem, also called the travelling salesperson problem or TSP, is a problem as follows. Given a list of cities and the distances between each pair of cities, we want to find the shortest possible route that visits each city *exactly once* and returns to the origin city.

Specifically, given an **undirected weighted graph** $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, which is an ordered pair of nodes set $\mathcal{E}$ and edges set $\mathcal{V} \subseteq \mathcal{E} \times \mathcal{E}$ where $\mathcal{G}$ is equipped with **spatial structure**. This means that each edge between nodes will have different weights and each node will have its coordinates, we want to find a simple cycle that visits every node exactly once while having the smallest cost.

Our approach is based on the application of graph neural networks. After generating the dataset, we will first use imitation learning to obtain a pretty good path, which will then be inputted into reinforcement learning to obtain the optimal solution. Since imitation learning converges very fast, we expect our run time to be promising compared to tradition exact solutions.

2 Proposed Method

2.1 Problem Formulation

2.1.1 TSP as Integer Linear Programming

We first formulate TSP in terms of **Integer Linear Programming**. Given an undirected weighted group $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, we label the nodes with numbers $1, \dots, n$ and define

$$x_{ij} := \begin{cases} 1, & \text{if } (i, j) \in \mathcal{E}' \\ 0, & \text{if } (i, j) \in \mathcal{E} \setminus \mathcal{E}', \end{cases}$$

where $\mathcal{E}' \subset \mathcal{E}$ is a variable which can be viewed as a compact representation of all variables $x_{ij}, \forall i, j$. Furthermore, we denote the weight on edge (i, j) by c_{ij} , then for a particular TSP problem instance, we can formulate the problem as follows.

$$\begin{aligned} \min \quad & \sum_{i=1}^n \sum_{j \neq i, j=1}^n c_{ij} x_{ij} : \\ & x_{ij} \in \{0, 1\} & i, j = 1, \dots, n; \\ & u_i \in \mathbb{Z} & i = 2, \dots, n; \\ & \sum_{i=1, i \neq j}^n x_{ij} = 1 & j = 1, \dots, n; \\ & \sum_{j=1, j \neq i}^n x_{ij} = 1 & i = 1, \dots, n; \\ & u_i - u_j + nx_{ij} \leq n - 1 & 2 \leq i \neq j \leq n; \\ & 1 \leq u_i \leq n - 1 & 2 \leq i \leq n. \end{aligned} \tag{1}$$

This is the Miller-Tucker-Zemlin formulation[11]. Note that in our case, since we're going to solve TSP exactly, hence all variables are integers, and we sometimes call this kind of integer linear programming as **pure integer programming**.

Since integer programming is a NP-Hard problem, there are no known polynomial algorithm can solve this explicitly. Hence, modern approach to such a problem is to **relax** the integrality constraint, which makes Equation 1 becomes a continuous linear programming (LP), whose solution provides a lower bound to Equation 1 since it's a relaxation, and we're trying to find the minimum.

Since an LP is a convex optimization problem, we have many polynomial time algorithms to solve the relaxed version. After obtaining a relaxed solution, if such LP relaxed solution respects the integrality constraint, we see that it's indeed a solution to Equation 1. But if not, we can simply divide the original relaxed LP into two sub-problems by **splitting the feasible region** according to a variable that does not respect integrality in the current relaxed LP solution x^* ,

$$x_i \leq \lfloor x_i^* \rfloor \vee x_i \geq \lceil x_i^* \rceil, \quad \exists i \leq p \mid x_i^* \notin \mathbb{Z}. \tag{2}$$

We see that by adding such additional constraint in two sub-problems respectively, we get a recursive algorithm called **Branch-and-Bound** [15].

2.1.2 Branching Rules

The branch-and-bound algorithm is widely used to solve integer programming problems. We see that the key step in the branch-and-bound algorithm is selecting a non-integer variable to **branch on** in Equation 2. And as one can expect, some choices may reduce the recursive searching tree significantly [2], hence the *branching rules* are the core of modern combinatorial optimization solvers, and it has been the focus of extensive research [10, 12, 5, 1]. There are several popular strategies [3] used in modern solver.

1. Strong branching. It'll result in the smallest recursive tree by computing the expected bound improvement for **each** candidate variable before branching by finding solutions of two LPs for every candidate, which is undoubtedly, very expensive. [4]

2. Conflict score.
3. Pseudo-cost. [12]
4. Hand-crafted combination of above.
5. Hybrid branching. Which basically computes a strong branching scores only at the beginning of the solving, and gradually switches to other methods mentioned above [3, 1].

2.2 Theoretical Framework

Our objective now is to learn strong branching strategy without expensive evaluation. And since this is a discrete time control process, hence we model the problem by Markov Decision Process (MDP) [8].

2.2.1 Markov Decision Process (MDP)

Given a regular Markov decision process $\mathcal{M} := (\mathcal{S}, \mathcal{A}, p_{\text{init}}, p_{\text{trans}}, R)$, where

- State space $\mathcal{S}$
- Action space $\mathcal{A}$
- Initial state distribution $p_{\text{init}}: \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$
- State transition distribution $p_{\text{trans}}: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$
- Reward function $R: \mathcal{S} \rightarrow \mathbb{R}$

We note that in the above definition of Markov Decision Process, the reward function R need not be deterministic. In other words, we can define R as a random function which will take a value based on a particular state in $\mathcal{S}$ with some randomness. Note that if R in $\mathcal{M}$ is equipped with any kind of randomness, we can write the reward r_t at time t as

$$r_t \sim p_{\text{reward}}(r_t \mid s_{t-1}, a_{t-1}, s_t),$$

and this can be converted into an equivalent Markov Decision Process $\mathcal{M}'$ with a deterministic reward function R' , where the randomness is integrated into parts of the states.

With an action policy $\pi: \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ such that the action a_t taken at time t is determined by

$$a_t \sim \pi(a_t \mid s_t),$$

we see that an MDP can be unrolled to produce a *trajectories* composed by state-action pairs as

$$\tau = (s_0, a_0, s_1, a_1, \dots)$$

which obeys the joint distribution

$$\tau \sim \underbrace{p_{\text{init}}(s_0)}_{\text{initial state}} \prod_{t=0}^{\infty} \underbrace{\pi(a_t \mid s_t)}_{\text{next action}} \underbrace{p_{\text{trans}}(s_{t+1} \mid a_t, s_t)}_{\text{next state}}.$$

2.2.2 Partially Observable Markov Decision Process (PO-MDP)

Follows the same idea from MDP, in the PO-MDP setting deals with the case that when the **complete** information about the current MDP state $\mathcal{S}$ is unavailable or not necessarily for the decision-making [16]. Instead, in our case, only a partial **observation** $o \in \Omega$ is available, where Ω is called the **partial state space**. We can use an active perspective to view the above model, namely we're merely applying a observation function $O: \mathcal{S} \rightarrow \Omega$ to the current state s_t at each time step t . Hence, we define a PO-MDP $\widetilde{\mathcal{M}}$ as a tuple

$$\widetilde{\mathcal{M}} := (\mathcal{S}, \mathcal{A}, p_{\text{init}}, p_{\text{trans}}, R, O).$$

Within this setup, a trajectory of PO-MDP takes form as

$$\tau = (o_0, r_0, a_0, o_1, r_1, a_1, \dots),$$

where $o_t := O(s_t)$ and $r_t := R(s_t)$. Specifically, noting that here r_t still depends on the state of the OP-MDP, not the observation.

We introduce a convenience variable $h_t: (o_0, r_0, a_0, \dots, o_t, r_t) \in \mathcal{H}$, which represents the PO-MDP history at time step t **without the action** a_t . Due to the non-Markovian nature of the trajectories,

$$o_{t+1}, r_{t+1} \not\perp h_{t-1} \mid o_t, r_t, a_t,$$

namely the decision-maker must take the whole history of observations, rewards and actions into account to decide on an optimal action at current time step t . We then see that action policy for PO-MDP takes the form

$$\tilde{\pi}: \mathcal{A} \times \mathcal{H} \rightarrow \mathbb{R}_{\geq 0}$$

such that $a_t \sim \pi(a_t \mid h_t)$.

2.3 Markov Control Problem

2.3.1 The MDP Control Problem

We define the MDP control problem as that of finding a policy $\pi^*: \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ which is optimal with respect to the expected total reward, that is,

$$\pi^* = \arg \max_{\pi} \lim_{T \rightarrow \infty} \mathbb{E}_{\tau} \left[\sum_{t=0}^T r_t \right],$$

where $r_t := R(s_t)$.

2.3.2 The PO-MDP Control Problem

Similar to the MDP control problem, the objective is to find a policy $\tilde{\pi}^*: \mathcal{A} \times \mathcal{H} \rightarrow \mathbb{R}_{\geq 0}$ such that it maximizes the expected total rewards, that is,

$$\tilde{\pi}^* = \arg \max_{\pi} \lim_{T \rightarrow \infty} \mathbb{E}_{\tau} \left[\sum_{t=0}^T r_t \right],$$

where $r_t := R(s_t)$.

2.4 Learning Pipeline

Since the branch-and-bound variable selection problem can be naturally formulated as a Markov decision process, hence a natural machine learning algorithm to use is reinforcement learning. [14] Specifically, since there are some state-of-the-art integers programming solvers out there, for examples, Gurobi¹, SCIP², etc, we decide to try imitating learning[9] by learning directly from an expert branching rule.

While there are some related works in this approach [6] which aims to tackle **mixed integer linear programming** (MILP), which means there are only some variables have integrality constraints, while other variables can be real numbers. But our approach try to extend this further, we're focusing on TSP, which not only is a pure integer programming, and the variables value is bounded to be in $\{0, 1\}$ only. Except all these, since imitating learning can't outperform SOTA solver in its nature, hence though the work in [6] is interesting, but we still think we can do better by focusing on a particular domain (TSP in this case) and try to formulate a problem specific reward function for TSP.

Hence, our proposed learning pipeline is as follows. Firstly, we only use imitating learning following [6] to obtain a good enough model, then we turn to unsupervised learning setup under the framework of reinforcement learning, which should make the network learn some domain-specific property and help to boost the performance.

2.4.1 Imitating Learning

As we mentioned before, the most effective branching strategy is strong branching, hence we train the network by behavioral cloning [13]. We first run the SOTA solver and get state-action pairs

$$\mathcal{D} = \{(s_i, \mathbf{a}_i^*)\}_{i=1}^N,$$

¹<https://www.gurobi.com/>

²<https://www.scipopt.org/>

and then learn our policy $\tilde{\pi}^*$ by minimizing the cross-entropy loss

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{(s, \mathbf{a}^*) \in \mathcal{D}} \log \tilde{\pi}_{\theta}(\mathbf{a}^* | s).$$

Additionally, we access (observe) the state of the branch-and-bound process by directly interacting with the SOTA solvers, specifically, SCIP. The implementation detail follows Gasse et al.[6].

2.4.2 Unsupervised Reinforcement Learning

After getting a stable model, we remove the expert’s instructions and let the model explore by itself since we’re focusing on TSP, which is a particular problem type in pure integer programming. That is, we are now going to define a new reward function $R: \mathcal{S} \rightarrow \mathbb{R}$.

We are still exploring this part, but some useful and observable states can be used in our formulation of R , including

1. Estimated branching tree sizes and its decreasing rate
2. Current tour length and its decreasing rate
3. etc.

And also some ideas about how should this function look like.

1. Reward decay forcing fast solving.
2. Traditional heuristics, e.g. k -opt [7].
3. Use additional information, like the optimal tour length.
4. Random reward for exploring.

In this way, from the success in [6], we’re guaranteed with a good enough model status, namely a good enough branching strategy after our first stage. Then, we do not need to worry about the technical detail in reinforcement learning like initial training but focus on giving a general pipeline for how to tackle a particular problem in this kind of combinatorial optimization problems.

Besides, there is a reason why we do not use pre-trained models from [6]. An obvious reason is that our focus is different: [6] focuses on general MILP, while we specifically focus on TSP (Pure IP with bounded variable values), which is a slightly but still different field.³

3 Related Work

There has been a lot of progress on the symmetric TSP in the last century, as it is one of the classical NP-hard combinatorial problems. With the increase in the number of nodes, there is a superpolynomial (at least exponential) explosion in the number of potential solutions. This makes the TSP problem difficult to solve on two parameters, the first being finding a global shortest route as well as reducing the computation complexity in finding this route. Concorde⁴, written in the ANSI C programming language, is widely recognized as the state-of-the-art(fastest) exact TSP solution for large datasets.

Our approach to solve TSP exactly is different from the Concorde approach, as well as the methods mentioned in the papers above. We will first use imitation training, which has been proven to converge fast, to obtain an approximate of the optimal path. Then, we will use our current path to initialize the reinforcement learning in order to obtain the global shortest path. We expect our combination of the two methods on GNN to give promising run time compared to other existing exact solvers.

(I will add some more explanations of the difference once I study our content for section 2 and 3.)

³Using the pre-trained models will be a good idea to see the generalizability in RL after we obtain a good enough reward function.

⁴<http://www.math.uwaterloo.ca/tsp/concorde/>

4 Experimental Results

Our implementation is based on Ecole⁵.

For our data, we will be using a data generator created from the Concorde TSP solver. This data generator will allow us to efficiently generate TSP problem sets of any size that we want; additionally, we can use their solver as well in order to first generate our training set for imitation learning. Once we have moved out of imitation learning, it will still be very useful to be able to have the ability to (relatively) quickly check our model's output for correctness.

Fortunately, as long as we design our model around the output of the Concorde TSP generator, we do not need any further pre-processing.

5 Future Milestone

1. Define an explicit reward function.
2. Using more sophisticated model in the imitating learning part.
3. Benchmarking all results from different exact solver.
4. Comparing the performance between using pre-trained model and not using one.

6 Conclusion

Author Contributions

7 Example to include figures

References

- [1] Tobias Achterberg and Timo Berthold. Hybrid branching. pages 309–311, 05 2009. ISBN 978-3-642-01928-9. doi: 10.1007/978-3-642-01929-6_23.
- [2] Tobias Achterberg and Roland Wunderling. *Mixed Integer Programming: Analyzing 12 Years of Progress*, pages 449–481. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013. ISBN 978-3-642-38189-8. doi: 10.1007/978-3-642-38189-8_18. URL https://doi.org/10.1007/978-3-642-38189-8_18.
- [3] Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Operations Research Letters*, 33(1):42–54, 2005. ISSN 0167-6377. doi: <https://doi.org/10.1016/j.orl.2004.04.002>. URL <https://www.sciencedirect.com/science/article/pii/S0167637704000501>.
- [4] D. Applegate, R. Bixby, V. Chvatal, and B. Cook. Finding cuts in the tsp (a preliminary report). Technical report, 1995.
- [5] Matteo Fischetti and Michele Monaci. Branching on nonchimerical fractionalities. *Operations Research Letters*, 40:159–164, 05 2012. doi: 10.1016/j.orl.2012.01.008.
- [6] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32*, 2019.
- [7] Keld Helsgaun. General k-opt submoves for the lin-kernighan tsp heuristic. *Mathematical Programming Computation*, 1:119–163, 10 2009. doi: 10.1007/s12532-009-0004-6.

⁵<https://github.com/ds4dm/ecole>

- [8] R.A. Howard. *Dynamic programming and Markov processes*. Technology Press of Massachusetts Institute of Technology, 1960. URL <https://books.google.com/books?id=FXJEAATAAJ>.
- [9] Ahmed Hussein, Mohamed Medhat Gaber, Eyad Elyan, and Chrisina Jayne. Imitation learning: A survey of learning methods. *ACM Comput. Surv.*, 50(2), apr 2017. ISSN 0360-0300. doi: 10.1145/3054912. URL <https://doi.org/10.1145/3054912>.
- [10] Jeff T. Linderoth and Martin W. P. Savelsbergh. A computational study of search strategies for mixed integer programming. *INFORMS J. Comput.*, 11(2):173–187, 1999. doi: 10.1287/ijoc.11.2.173. URL <https://doi.org/10.1287/ijoc.11.2.173>.
- [11] C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation of traveling salesman problems. *J. ACM*, 7(4):326–329, oct 1960. ISSN 0004-5411. doi: 10.1145/321043.321046. URL <https://doi.org/10.1145/321043.321046>.
- [12] Jagat Patel and John Chinneck. Active-constraint variable ordering for faster feasibility of mixed integer linear programs. *Math. Program.*, 110:445–474, 09 2007. doi: 10.1007/s10107-006-0009-0.
- [13] Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991. doi: 10.1162/neco.1991.3.1.88.
- [14] R.S. Sutton and A.G. Barto. *Reinforcement Learning, second edition: An Introduction*. Adaptive Computation and Machine Learning series. MIT Press, 2018. ISBN 9780262352703. URL <https://books.google.com/books?id=uWVODwAAQBAJ>.
- [15] Laurence Wolsey. *Branch and Bound*, chapter 7, pages 113–138. John Wiley and Sons, Ltd, 2020. ISBN 9781119606475. doi: <https://doi.org/10.1002/9781119606475.ch7>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119606475.ch7>.
- [16] K.J Åström. Optimal control of markov processes with incomplete state information. *Journal of Mathematical Analysis and Applications*, 10(1):174–205, 1965. ISSN 0022-247X. doi: [https://doi.org/10.1016/0022-247X\(65\)90154-X](https://doi.org/10.1016/0022-247X(65)90154-X). URL <https://www.sciencedirect.com/science/article/pii/0022247X6590154X>.